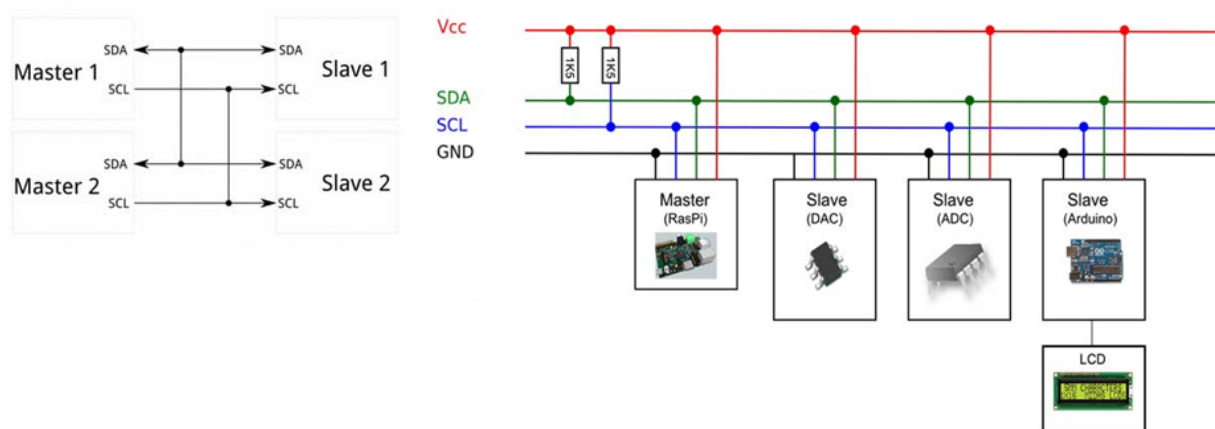


I²C 16

I²C (i-square-c) مخفف "Inter-Integrated-Circuit" است که در اصل توسط شرکت فیلیپس نیمه‌هادی‌ها (اکنون NXP) در سال 1982 ایجاد شد. I²C™ یک علامت تجاری ثبت‌شده برای مالک مربوطه‌اش است و ممکن است دلیل اینکه در برخی میکروکنترلرها مانند Atmel AVR به آن "Two Wire Interface (TWI)" می‌گویند، همین باشد. I²C یک باس ارتباطی سریال چند مستر، چند اسلیو، همزمان، دوطرفه و نیمه‌دوپلکس است که به طور گسترده‌ای برای اتصال ICهای جانبی با سرعت پایین به پردازنده‌ها و میکروکنترلرها در ارتباطات کوتاه‌مدت و درون‌بردی استفاده می‌شود.

این رابط می‌تواند برای مقاصد مختلفی از جمله تولید و تأیید CRC، SMBus (باس مدیریت سیستم) و PMBus (باس مدیریت انرژی) استفاده شود. بسته به پیاده‌سازی خاص دستگاه، قابلیت DMA برای کاهش بار CPU ممکن است در دسترس باشد.

در این پروتکل تعدادی مستر و اسلیو به باس متصل می‌شوند. نمایی از اتصالات باس در شکل نشان داده شده است. خطوط داده و کلاک بایستی از مقاومت‌های حدود 1.5 تا 4.7 کیلو اهمی به Vcc متصل گردند تا در شرایطی که باس بیکار هست مقدار یک منطقی را داشته باشد.



نمایی از ارتباط I²C

16.1 ویژگی‌های اصلی I²C

- مبدل پروتکل I²C/باس موازی

- قابلیت چند مستر : یک رابط می‌تواند به عنوان مستر یا اسلیو عمل کند.

- ویژگی‌های مستر I^2C :

- تولید کلاک

- تولید شرایط شروع و توقف

- ویژگی‌های اسلیو I^2C :

- تشخیص آدرس I^2C قابل برنامه‌ریزی

- قابلیت آدرس‌دهی دوگانه برای تأیید 2 آدرس اسلیو

- تشخیص بیت توقف

- تولید و تشخیص آدرس‌دهی 7 بیتی/10 بیتی و تماس عمومی

- پشتیبانی از سرعت‌های مختلف ارتباط

- پرچم‌های وضعیت :

- پرچم حالت فرستنده/گیرنده

- پرچم پایان انتقال بایت

- پرچم مشغول بودن I^2C

- پرچم‌های خطا :

- شرایط از دست رفتن یک اسلیو که آدرس دهی شده در حالت مستر

- عدم تأیید پس از انتقال آدرس/داده

- تشخیص شرایط شروع یا توقف نادرست

- Overrun/Underrun اگر کشش کلاک غیرفعال باشد

- 2 وکتور وقفه :

- 1 وقفه برای ارتباط موفق آدرس/داده

- 1 وقفه برای شرایط خطا

- کشش اختیاری کلاک

- بافر 1 بایتی با قابلیت DMA

- تولید یا تأیید قابل تنظیم PEC (بررسی خطای بسته):

- سازگاری با SMBus 2.0

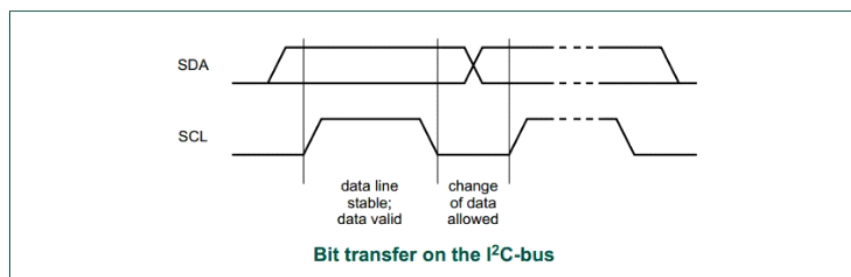
- سازگاری با PMBus

16.2 لایه فیزیکی I²C (سخت‌افزار)

باس I²C دارای خروجی open-drain یا open-collector برای خطوط SDA و SCL می‌باشد. لذا با یک مقاومت به Vcc متصل شده است. به همین دلیل، حالت پیش‌فرض (IDLE) برای هر خط زمانی که درایور open-drain خاموش است، HIGH است. با این حال، اگر درایور خروجی را روشن کنیم، پین ورودی/خروجی توسط ترانزیستور درایور خروجی به زمین (LOW) کشیده می‌شود، همان‌طور که در دیاگرام زیر مشاهده می‌کنید.

خطوط باس I²C که پین‌های دوطرفه "open-drain" هستند، آن را برای ارتباطات چند مستر و چند برده ایده‌آل می‌سازد. هنگامی که چندین مستر روی باس قرار دارد هر مستری که سریعتر پیام شروع (معادل صفر) را ارسال نماید باس را در اختیار گرفته و سایر مسترها تا پایان ارتباط به باس دسترسی ندارند. چگونگی تخصیص باس به مسترها یک ویژگی بسیار قدرتمند برای ارتباط I²C است.

ارتباط I²C از طریق دو خط SDA و SCL، برقرار می‌شود. هر دو خط SCL و SDA خطوط دوطرفه هستند که از طریق یک منبع جریان یا مقاومت Pull-up به ولتاژ مثبت متصل شده‌اند. زمانی که باس آزاد است، هر دو خط HIGH هستند. نحوه ارسال بیتها در باس I²C نشان داده شده است.



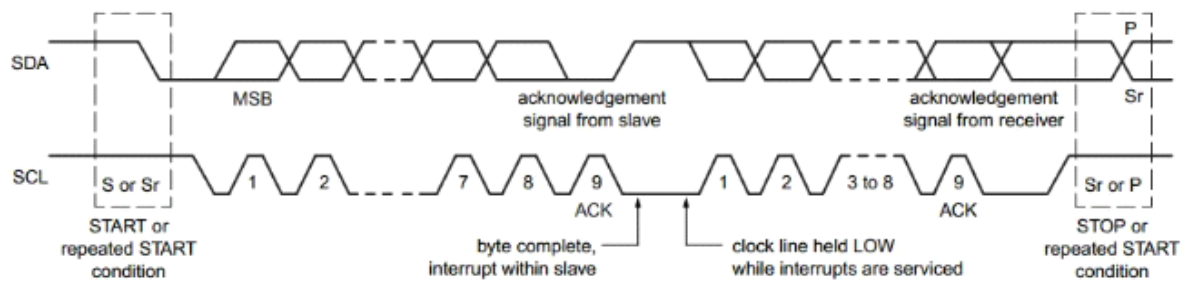
16.3 فرمت فریم I²C

یک پیام معمولی I²C از چند بخش تشکیل شده است که به طور متوالی در باس I²C اتفاق می‌افتد و همیشه با یک شرط شروع (s) آغاز می‌شود. سپس آدرس دستگاه برده مورد نظر (7 بیت یا 10 بیت) ارسال می‌شود، و بعد از آن یک بیت R/W برای تعیین اینکه آیا مستر (که شرایط S را برای ارتباط آغاز کرده است) می‌خواهد به این اسلیو با آن آدرس بخواند یا بنویسد، قرار می‌گیرد.

اگر اسلیو وجود داشته باشد و به درستی کار کند، به مستر با ارسال یک بیت تأیید (ACK) پاسخ می‌دهد، در غیر این صورت به عنوان تأیید منفی (NACK) در نظر گرفته می‌شود. پس از آن، بایت داده ارسال می‌شود و اسلیو دوباره تأیید می‌کند. در نهایت، مستر می‌تواند ارتباط را با ارسال توالی شرط توقف (P) خاتمه دهد.

هر فریم از بخش‌های ذیل تشکیل شده است که در شکل هم نشان داده شده است.

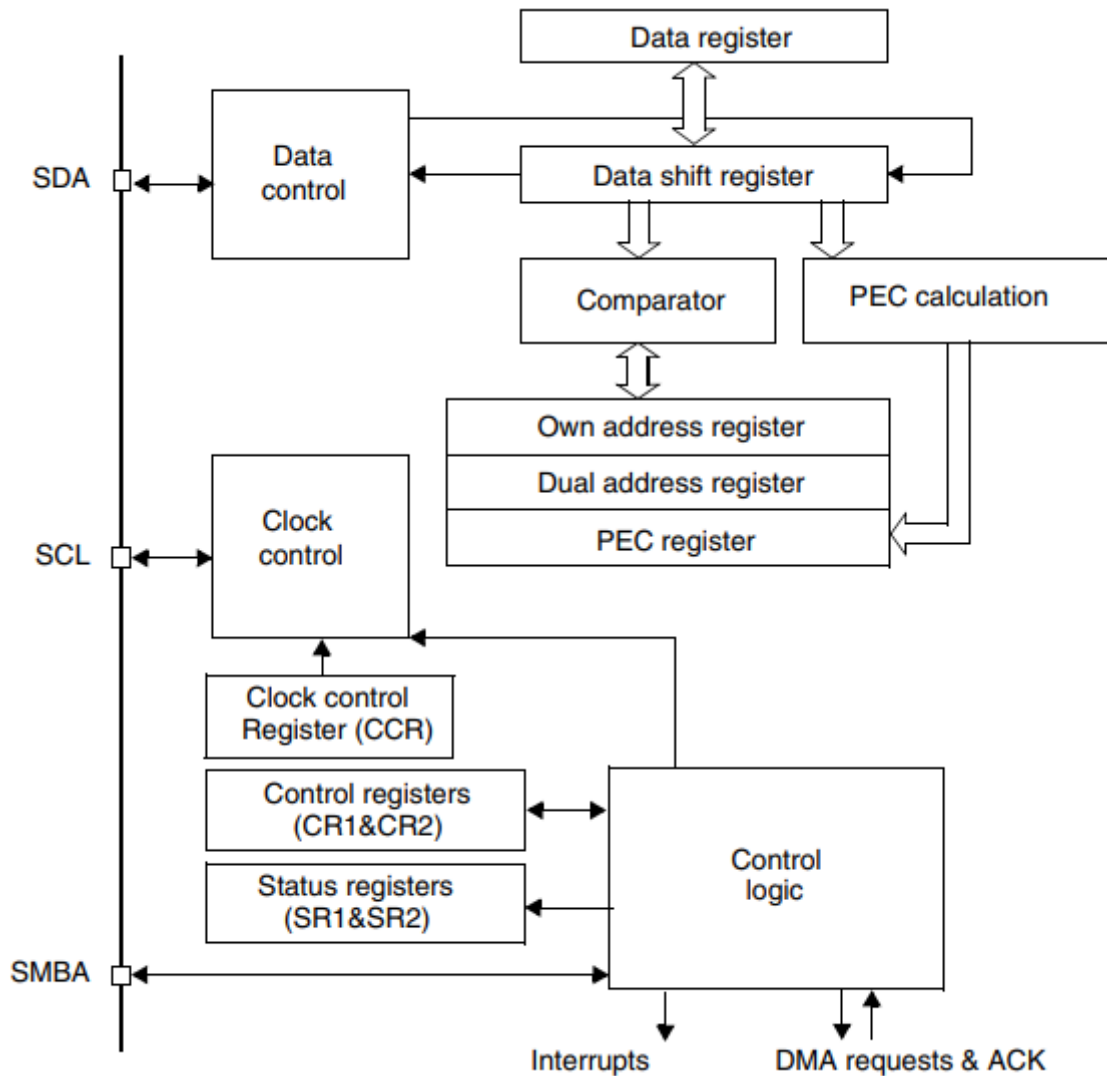
- Start Condition (S) - شرط شروع (S): علامت آغاز تراکنش.
- Stop Condition (P) - شرط توقف (P): علامت پایان تراکنش.
- Repeated Start (Restart) Condition (Sr) - شرط شروع تکراری (Sr): علامتی برای شروع مجدد تراکنش بدون توقف.
- Acknowledge ACK (A) - تأیید (A) (ACK): علامتی که نشان‌دهنده تأیید دریافت داده توسط برده است.
- Not Acknowledge NACK (~A) - عدم تأیید (~A) (NACK): علامتی که نشان‌دهنده عدم تأیید دریافت داده توسط برده است.
- Address + R/W - آدرس + R/W: ترکیبی از آدرس دستگاه اسلیو و بیت خواندن/نوشتن.
- Data Byte - بایت داده: داده‌ای که بین مستر و برده منتقل می‌شود.



Data transfer on the I²C-bus

16.4 بلوک دیاگرام I²C

در شکل نمایی از بلوک دیاگرام I²C که در میکرو پیاده سازی شده است نمایش داده شده است.



همان‌طور که در دیاگرام بلوکی I²C در بالا مشاهده می‌کنید، یک رجیستر اصلی، یک رجیستر بافر و برخی واحدهای منطقی کنترل برای مدیریت تمام مراحل تراکنش I²C وجود دارد. این مراحل شامل بررسی تطابق آدرس، تولید سیگنال ساعت، فیلتر کردن، بررسی خطا و غیره است.

16.5 سرعت‌های باس I²C

در ابتدا، باس I²C به عملیات با سرعت 100 کیلوبیت در ثانیه محدود بود. با گذشت زمان، چندین تغییر به مشخصات اضافه شده است به طوری که اکنون پنج دسته سرعت عملیاتی وجود دارد. دستگاه‌های حالت استاندارد، حالت سریع (Fm)، حالت سریع پلاس (+Fm) و حالت سرعت بالا (Hs-mode) با یکدیگر سازگار هستند. این بدان معناست که هر دستگاهی می‌تواند با سرعت باس کمتری عمل کند. دستگاه‌های حالت فوق سریع با نسخه‌های قبلی سازگار نیستند زیرا باس یک‌طرفه است.

باس دوطرفه:

- حالت استاندارد (Sm) با نرخ بیت تا 100 کیلوبیت در ثانیه
- حالت سریع (Fm) با نرخ بیت تا 400 کیلوبیت در ثانیه
- حالت سریع پلاس (+Fm) با نرخ بیت تا 1 مگابیت در ثانیه
- حالت سرعت بالا (Hs-mode) با نرخ بیت تا 3.4 مگابیت در ثانیه

باس یک‌طرفه:

- حالت فوق سریع (UFm) با نرخ بیت تا 5 مگابیت در ثانیه
- هر دستگاه متصل به باس می‌تواند در یکی از چهار حالت زیر عمل کند:

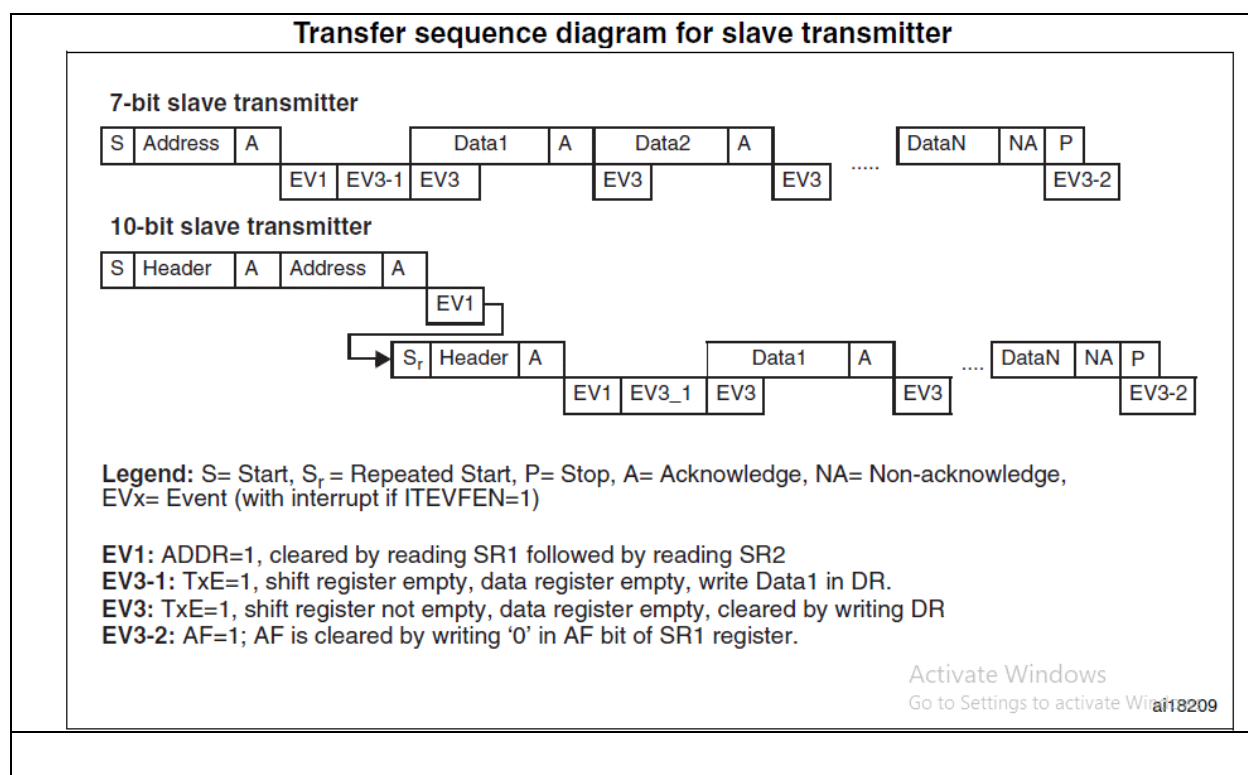
- فرستنده اسلیو
- دریافت‌کننده اسلیو
- فرستنده مستر
- دریافت‌کننده مستر

16.6 اسلیو در I²C

به‌طور پیش‌فرض، رابط I²C در حالت اسلیو (Slave) کار می‌کند. به محض تشخیص یک شرط شروع، آدرس از خط SDA دریافت شده و به ثبت شیفت‌ر ارسال می‌شود. سپس این آدرس با آدرس خودش به عنوان اسلیو مقایسه میکند اگر یکسان باشد با ارسال A یا پیام تایید به ارتباط ادامه میدهد و اگر آدرس خودش نباشد باس را رها میکند و منتظر دریافت پیام شروع بعدی میماند.

16.6.1 اسلیو به عنوان فرستنده

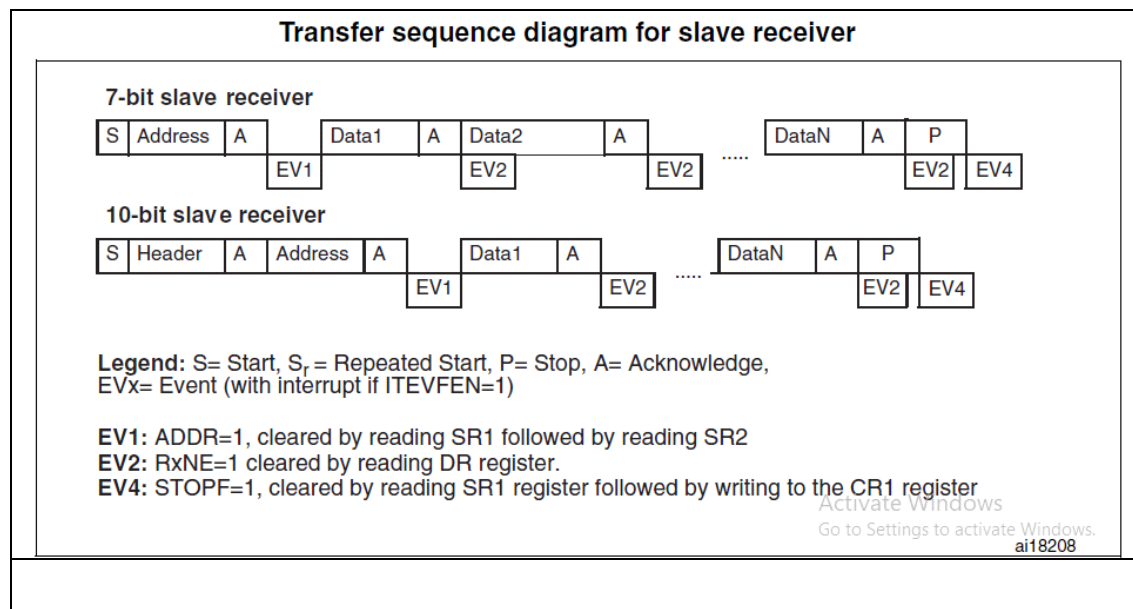
هنگامی که مستر آدرس اسلیو را ارسال کرد و وضعیت اسلیو را فرستنده در نظر بگیرد، اسلیو داده ها را روی باس قرار میدهد. سپس مستر به ازای داده دریافتی پالس تایید یا A را ارسال میکند. و در ادامه اسلیو داده های بعدی را تا زمانی که مستر پیام A را ارسال کند میفرستد. اگر مستر پیام A ارسال نکند یا به عبارتی NA ارسال شود به منزله پایان دریافت توسط مستر یا رخداد خطا هست و اسلیو دیگر داده ای را ارسال نخواهد کرد و مستر ارتباط را متوقف میکند. جزئیات این ارتباط در شکل نشان داده شده است.



16.6.2 اسلیو به عنوان دریافت کننده

هنگامی که مستر آدرس اسلیو را ارسال کرد و وضعیت اسلیو را گیرنده در نظر بگیرد، مستر داده ها را روی باس قرار میدهد. سپس اسلیو به ازای داده دریافتی پالس تایید یا A را ارسال میکند. و در ادامه مستر داده های بعدی را ارسال میکند و تا هر زمانی که مستر بخواهد اطلاعات ارسال میشود. و نهایتاً مستر ارتباط را خاتمه میدهد. اگر

اسلیو پیام A را ارسال نکند مستر ارتباط را متوقف کرده و وقفه خطا تولید می‌شود. جزئیات این ارتباط در شکل نشان داده شده است.



پس از انتقال آخرین بایت داده، شرط توقف (Stop Condition) توسط مستر تولید می‌شود. اسلیو این شرط را شناسایی کرده و ارتباط را خاتمه داده و باس را رها می‌کند و مجدد منتظر پیام شروع می‌ماند.

16.7 مستر در I²C

هر دستگاهی که روی باس I²C قرار داشته باشد به صورت پیشفرض در حالت اسلیو قرار دارد و چنانچه قابلیت ارسال پیام start داشته باشد، میتواند در نقش مستر ظاهر شده و باس را در اختیار بگیرد و یک انتقال داده را آغاز کرده و سیگنال کلاک را تولید نماید. یک انتقال داده همیشه با یک شرط شروع آغاز شده و با یک شرط توقف پایان می‌یابد.

مستر با ارسال start باس را در اختیار می‌گیرد. اگر روی باس چندین مستر وجود داشته باشد هر مستری که سریعتر پیام شروع را ارسال کند باس را در اختیار می‌گیرد و سایر مسترها تا زمانی که باس به حالت بیکار دربیاید منتظر می‌مانند.

16.7.1 شرط شروع

تنظیم بیت START باعث می‌شود دستگاه یک شرط شروع تولید کرده و دستگاه در نقش مستر ظاهر گردد. چنانچه در حالت مستر باشد و دوباره بیت START تنظیم شود، یک شرط شروع تکراری در پایان انتقال بایت فعلی تولید می‌شود. پس از ارسال شرط شروع، مستر آدرس اسلیو همراه با وضعیت فرستنده/گیرنده بودن اسلیو را ارسال می‌کند.

16.7.2 انتقال آدرس اسلیو

آدرس اسلیو به خط SDA از طریق ثبت شیفر داخلی ارسال می‌شود. آدرس دهی در دو مد 7 بیتی و ده بیتی انجام می‌شود. در مد 7 بیتی، یک بایت شامل 7 بیت آدرس و بیت کم ارزش حاوی وضعیت فرستنده/گیرنده اسلیو ارسال می‌گردد. اگر این بیت وضعیت برابر با صفر باشد یعنی اسلیو در حالت گیرنده و مستر فرستنده است و اگر یک باشد، اسلیو در حالت فرستنده و مستر گیرنده خواهد بود.

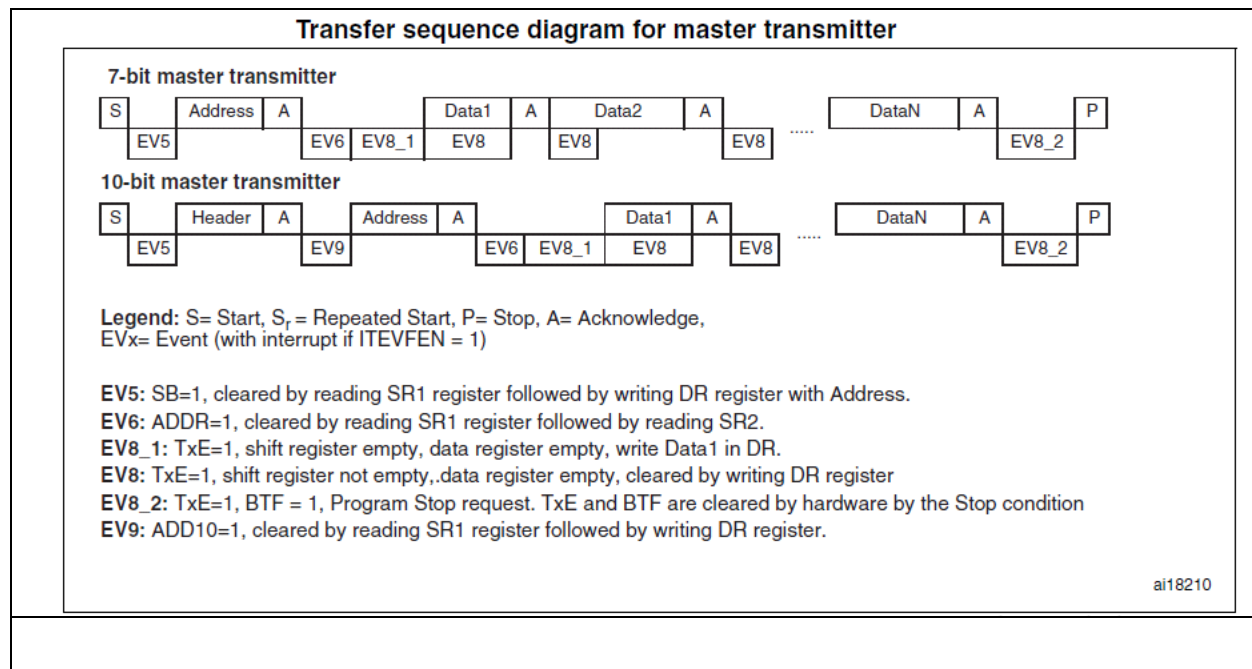
در حالت ده بیتی، آدرس توسط دو بایت ارسال می‌گردد که بایت اول هدر نامیده می‌شود و شامل کد رزرو و دو بیت با ارزش آدرس اسلیو و بیت فرستنده/گیرنده اسلیو است. و بایت دوم شامل 8 بیت آدرس کم ارزشتر اسلیو می‌باشد. برای وارد شدن به حالت فرستنده برای مستر و گیرنده برای اسلیو، مستر عبارت 11110xx0 را ارسال کرده و سپس آدرس اسلیو را ارسال می‌کند. (xx دو بیت با ارزش آدرس است).

برای وارد شدن به حالت دریافت‌کننده برای مستر و حالت فرستنده برای اسلیو، مستر عبارت 11110xx0 را ارسال کرده و سپس آدرس اسلیو را ارسال می‌کند. سپس باید یک شرط شروع تکراری ارسال کند که به دنبال آن عبارت 11110xx1 باشد.

16.7.3 ارسال داده توسط مستر

پس از انتقال آدرس فرستنده (Master) بایت‌های داده را از رجیستر DR به خط SDA منتقل می‌کند. پس از ارسال هر داده منتظر پالس تأیید یا همان A می‌ماند و پس از دریافت A سپس داده بعدی را ارسال می‌کند. اگر به ازای هر داده، A دریافت شود به پروسه ادامه می‌دهد و اگر A دریافت نشد ارسال داده را خاتمه داده و وقفه خطا رخ می‌دهد. جزئیات این پروسه در شکل نشان داده شده است.

در حالتی که مستر فرستنده است پس از آخرین ارسال داده، مستر وضعیت توقف یا stop را تنظیم کرده و سپس باس را رها میکند.



16.7.4 دریافت داده توسط مستر

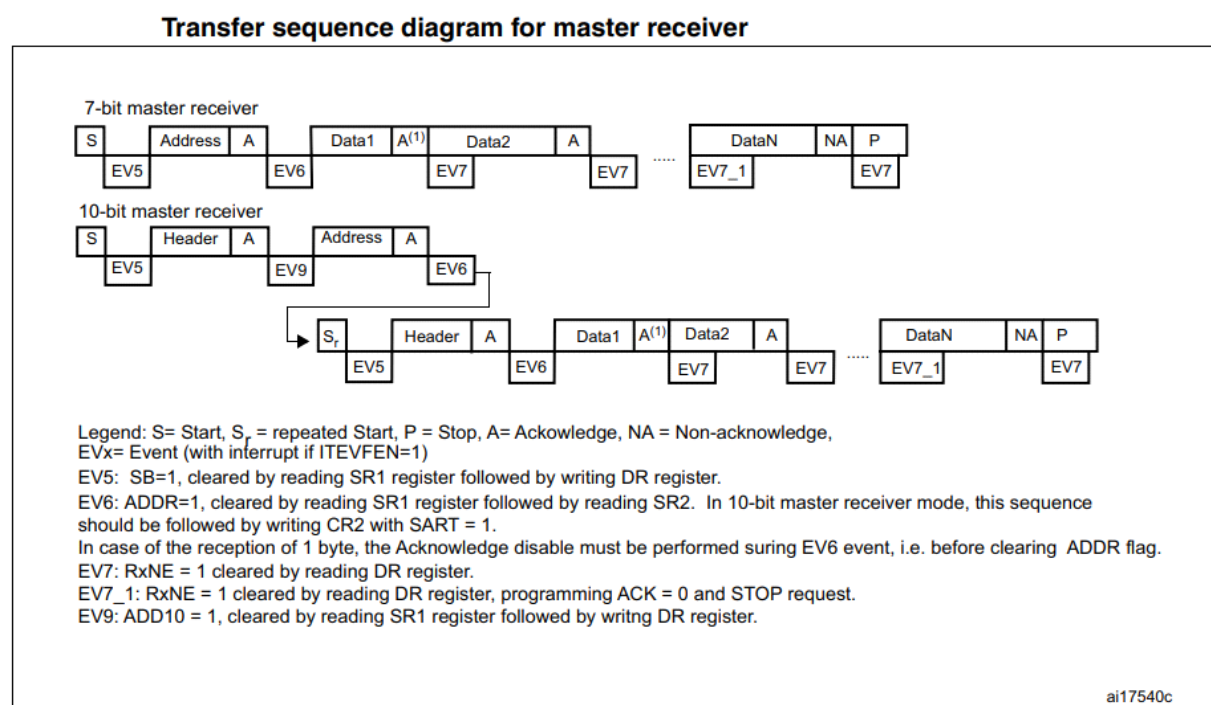
پس از انتقال آدرس و مشخص کردن وضعیت اسال/دریافت داده توسط مستر ، چنانچه اسلیو فعال باشد داده ها را به صورت بایت به بایت روی باس قرارمی دهد . داده های ارسالی توسط مستر برداشت میشود و مستر میتواند به ازای هر دریافت داده پیام تصدیق یا ACK برای اسلیو ارسال کند. اسلیو هم تا زمانی که پیام ACK دریافت میکند داده جدیدی را ارسال مینماید.

فرستنده (Master) یک NACK برای آخرین بایت دریافتی از اسلیو می فرستد. پس از دریافت این NACK، اسلیو داده ای نمی فرستد و کنترل خطوط SCL و SDA را آزاد می کند. در این حالت مستر میتواند پیام stop را ارسال کند و ارتباط خاتمه پیدا کند. چنانچه مستر بخواهد از حالت گیرنده به فرستنده تغییر وضعیت دهد بایستی مجدد پیام start repeted ارسال نماید و ارسال را طبق فرایند توضیح داده شده در بخش قبل ادامه دهد.

پیام start repeted و start repeted هر دو از نظر تغییر سطح سیگنال از یک به صفر مشابه هستند ولی به لحاظ زمان بندی متفاوت هست. Start زمانی بکار میرود که باس آزاد است و یک مستر مایل است باس را در اختیار بگیرد. ولی پیام start repeted زمانی استفاده میشود که باس در اختیار یک مستر هست و مستر مایل است نقش خود

را از فرستنده به گیرنده و بالعکس تغییر دهد و در این حالت نیازی نیست باس آزاد شود و لذا این پیام به لحاظ زمانی کوتاهتر است.

فرآیند دریافت داده در شکل نشان داده شده است.



16.8 شرایط خطای I2C در STM32

برخی از شرایط خطا وجود دارند که می‌توانند توسط سخت‌افزار رابط I²C شناسایی شوند تا مشکلاتی را در سطح سخت‌افزار نشان دهند. نرم‌افزار می‌تواند به راحتی این شرایط خطا را با خواندن بیت‌های علامت مربوط به هر سیگنال خطا شناسایی کند. شرایط خطا شامل موارد زیر است:

خطای باس (BERR) – این خطا زمانی رخ می‌دهد که رابط I²C یک وضعیت توقف یا شروع خارجی را طی یک انتقال آدرس یا داده شناسایی کند.

عدم تأیید (AF) – این خطا زمانی رخ می‌دهد که رابط یک بیت عدم تأیید را شناسایی کند.

از دست دادن داوری (ARLO) – Arbitration Lost این خطا زمانی رخ می‌دهد که رابط I2C یک وضعیت Arbitration Lost را شناسایی کند. "Arbitration" فرایندی است که مشخص میکند کدام دستگاه اجازه دارد تا به خط دسترسی پیدا کند و در صورت تداخل، کدام دستگاه در اولویت است. این وضعیت معمولاً زمانی رخ می‌دهد که یک دستگاه در حال ارسال داده است، اما دیگر دستگاه‌ها نیز درخواست دسترسی به خط I2C دارند. در این صورت، اگر دستگاه دیگر بتواند خط را در اختیار بگیرد، دستگاهی که درگیر تبادل اطلاعات است متوجه می‌شود که نمی‌تواند به ارسال داده ادامه دهد و این وضعیت به عنوان "Arbitration Lost" شناخته می‌شود.

خطای اضافه‌بار/کم‌بار (OVR) – Overrun/Underrun Error یک خطای اضافه‌بار در حالت اسلیو (Slave mode) زمانی رخ می‌دهد که در حال دریافت داده باشد. و چنانچه یک بایت داده را دریافت کرده ولی هنوز برداشت نشده باشد و داده جدیدی منتظر دریافت باشد، خطای Overrun رخ می‌دهد. خطای کم‌بار می‌تواند در حالت اسلیو زمانی رخ دهد که داده‌ای ارسال شده باشد و باس منتظر داده بعدی برای ارسال باشد ولی هنوز از جانب اسلیو در رجیستر قرار نگرفته است.

16.9 وقفه‌های I²C در STM32

رویدادهای وقفه I²C به یک وکتور وقفه مشابه متصل هستند. بنابراین I²C یک سیگنال وقفه واحد صرف‌نظر از منبع آن ارسال می‌کند، و نرم‌افزار باید آن را شناسایی کند و به callback های مربوطه ارجاع دهد. این رویدادها در صورتی که بیت کنترل فعال‌سازی مربوطه تنظیم شده باشد، یک وقفه تولید می‌کنند.

I²C Interrupt requests

Interrupt event	Event flag	Enable control bit
Start bit sent (Master)	SB	ITEVFEN
Address sent (Master) or Address matched (Slave)	ADDR	
10-bit header sent (Master)	ADD10	
Stop received (Slave)	STOPF	
Data byte transfer finished	BTF	
Receive buffer not empty	RxNE	ITEVFEN and ITBUFEN
Transmit buffer empty	TxE	
Bus error	BERR	ITERREN
Arbitration loss (Master)	ARLO	
Acknowledge failure	AF	
Overrun/Underrun	OVR	
PEC error	PECERR	
Timeout/Tlow error	TIMEOUT	
SMBus Alert	SMBALERT	

16.10 روش های انتقال داده

16.10.1 Polling با I²C

نخستین و آسان‌ترین راه انجام هر کار در نرم‌افزارهای توکار، استفاده از polling برای بررسی منابع سخت‌افزاری تا زمانی است که آماده حرکت به مرحله بعدی در دستورات برنامه‌تان شود. با این حال، این روش کمترین کارایی را دارد و پردازنده به‌طور مداوم زمان زیادی را در حالت "مشغول به انتظار" هدر می‌دهد.

این وضعیت برای هر دو ارتباط ارسال و دریافت یکسان است. شما فقط منتظر می‌مانید تا بایت فعلی داده‌ای که باید ارسال شود، منتقل گردد تا بتوانید بایت بعدی را شروع کنید و به این صورت ادامه دهید.

16.10.2 Interrupts با I²C

اما، ما می‌توانیم Interruptهای I²C را فعال کنیم تا سیگنالی دریافت کنیم که هنگام اتمام و آماده بودن برای سرویس‌دهی توسط CPU را نشان دهد. این موضوع برای داده‌هایی که ارسال یا دریافت شده‌اند، خیلی مفید است

و باعث صرفه‌جویی زیادی در زمان می‌شود و همواره بهترین روش برای مدیریت چنین رویدادهایی محسوب می‌گردد.

با این حال، در برخی از برنامه‌های "حساستر به زمان"، نیاز داریم تا همه چیز به‌طور قطعی و با زمانبندی مشخص صورت گیرد. یکی از مشکلات اصلی Interruptها این است که نمی‌توانیم زمان وقوع آن را پیش‌بینی کنیم و نمی‌دانیم که در کدام کار (task) رخ می‌دهد. این موضوع می‌تواند باعث مختل شدن رفتار زمانی سیستم شود.

16.10.3 I²C با DMA

برای کار در حداکثر سرعت، I²C نیاز دارد که با داده‌ها برای ارسال تغذیه شود و داده‌های دریافتی در بافر Rx باید خوانده شوند تا از وقوع Overrun جلوگیری شود. برای تسهیل انتقالات، I²C ویژگی DMA را داراست که یک پروتکل درخواست/تأیید ساده را پیاده‌سازی می‌کند.

درخواست‌های DMA هنگامی تولید می‌شوند که رجیستر داده در حین ارسال خالی شود و در حین دریافت پُر گردد. استفاده از DMA همچنین CPU را از انجام انتقالات داده از لوازم جانبی به حافظه آزاد می‌کند. این موضوع در نهایت زمان زیادی را صرفه‌جویی کرده و به‌عنوان کارآمدترین روش برای مدیریت این انتقال داده از حافظه به لوازم جانبی و بالعکس در نظر گرفته می‌شود.

16.11 توابع hal برای I²C

در حالت کلی مجموعه توابعی که برای I²C توسط کتابخانه HAL منتشر شده برای انتقال داده به روش pollig ، وقفه و DMA عبارتند از:

16.11.1 توابع polling Mode

polling Mode: این توابع در حالت polling کار می‌کنند، به این معنی که تا زمانی که عملیات کامل نشود، میکروکنترلر در این توابع متوقف می‌شود. این می‌تواند باعث مسدود شدن برنامه در صورت بروز خطا یا زمان انتظار طولانی شود.

Timeout: استفاده از پارامتر Timeout به شما این امکان را می‌دهد که از مسدود شدن دائمی جلوگیری کنید و در صورت عدم موفقیت، بتوانید کنترل بیشتری بر روی برنامه داشته باشید

HAL_I2C_Master_Transmit

```
1 HAL_I2C_Master_Transmit (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t* pData, uint16_t Size, uint32_t Timeout);
```

شرح: این تابع برای ارسال داده‌ها از طرف میکروکنترلر (به عنوان ماستر) به یک دستگاه I2C (به عنوان اسلَو) استفاده می‌شود.

پارامترها:

hi2c: اشاره‌گر به ساختار I2C که تنظیمات و وضعیت ارتباط I2C را نگه می‌دارد.

DevAddress: آدرس دستگاه I2C که داده‌ها به آن ارسال می‌شود.

pData: اشاره‌گر به آرایه‌ای از داده‌ها که باید ارسال شوند.

Size: تعداد بایت‌هایی که باید ارسال شوند.

Timeout: زمان انتظار برای اتمام عملیات (به میلی‌ثانیه).

HAL_I2C_Master_Receive

```
1 HAL_I2C_Master_Receive (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t* pData, uint16_t Size, uint32_t Timeout);
```

شرح: این تابع برای دریافت داده‌ها از یک دستگاه I2C (به عنوان اسلَو) به میکروکنترلر (به عنوان ماستر) استفاده می‌شود.

پارامترها مشابه تابع قبلی است و عملکرد آن برعکس است، یعنی داده‌ها از دستگاه I2C به میکروکنترلر منتقل می‌شوند.

HAL_I2C_Slave_Transmit

```
1 HAL_I2C_Slave_Transmit (I2C_HandleTypeDef * hi2c, uint8_t* pData, uint16_t Size, uint32_t Timeout);
```

شرح: این تابع برای ارسال داده‌ها از طرف میکروکنترلر (به عنوان اسلَو) به یک دستگاه I2C (به عنوان ماستر) استفاده می‌شود.

پارامترها:

hi2c: اشاره گر به ساختار I2C.

pData: اشاره گر به آرایه ای از داده ها که باید ارسال شوند.

Size: تعداد بایت هایی که باید ارسال شوند.

Timeout: زمان انتظار برای اتمام عملیات.

HAL_I2C_Slave_Receive

¹ HAL_I2C_Slave_Receive (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size, uint32_t Timeout);

شرح: این تابع برای دریافت داده ها از یک دستگاه I2C (به عنوان ماستر) به میکروکنترلر (به عنوان اسلَو) استفاده می شود.

پارامترها مشابه تابع قبلی است و داده ها از ماستر به اسلَو منتقل می شوند.

HAL_I2C_Mem_Write

¹ HAL_I2C_Mem_Write (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size, uint32_t Timeout);

شرح: این تابع برای نوشتن داده ها به یک آدرس خاص در حافظه یک دستگاه I2C استفاده می شود.

پارامترها:

hi2c: اشاره گر به ساختار I2C.

DevAddress: آدرس دستگاه I2C.

MemAddress: آدرس حافظه داخلی دستگاه I2C که داده ها باید در آن نوشته شوند.

MemAddSize: اندازه آدرس حافظه (معمولاً ۱ یا ۲ بایت).

pData: اشاره گر به داده هایی که باید نوشته شوند.

Size: تعداد بایت هایی که باید نوشته شوند.

Timeout: زمان انتظار برای اتمام عملیات.

HAL_I2C_Mem_Read

```
1 HAL_I2C_Mem_Read (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t  
    MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size, uint32_t Timeout);
```

شرح: این تابع برای خواندن داده‌ها از یک آدرس خاص در حافظه یک دستگاه I2C استفاده می‌شود.

پارامترها مشابه تابع قبلی است و عملکرد آن برعکس است، یعنی داده‌ها از حافظه دستگاه I2C به میکروکنترلر منتقل می‌شوند.

16.11.2 توابع وقفه

HAL_I2C_Master_Transmit_IT

```
HAL_I2C_Master_Transmit_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t *  
    pData, uint16_t Size);
```

.

```
void HAL_I2C_MasterTxCpltCallback (I2C_HandleTypeDef * hi2c)  
{  
    // TX Done .. Do Something!  
}
```

پس از فراخوانی تابع فوق، دستگاه جانبی I2C شروع به ارسال تمام بایت‌های داده در بافر یک به یک می‌کند تا زمانی که کار تمام شود. وقتی این کار انجام شد، این تابع زیر فراخوانی و اجرا می‌شود، اگر می‌خواهید پس از تکمیل انتقال داده کاری انجام دهید، آن را به کد خود در فایل منبع برنامه (main.c) اضافه کنید

HAL_I2C_Master_Receive_IT

```
HAL_I2C_Master_Receive_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size);
```

```
void HAL_I2C_MasterRxCpltCallback (I2C_HandleTypeDef * hi2c)  
{  
    // RX Done .. Do Something!  
}
```

مشابه این دستورات برای اسلیو هم وجود دارد

پس از فراخوانی تابع فوق، دستگاه جانبی I2C شروع به دریافت تمام بایت های داده ورودی در بافر می کند تا زمانی که کار تمام شود. وقتی این کار انجام شد، این تابع زیر فراخوانی و اجرا می شود، اگر می خواهید پس از تکمیل دریافت داده کاری انجام دهید، آن را به کد خود در فایل منبع برنامه (main.c) اضافه کنید.

16.11.3 توابع DMA

HAL_I2C_Master_Transmit_DMA

```
HAL_I2C_Master_Transmit_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size);
```

```
void HAL_I2C_MasterTxCpltCallback (I2C_HandleTypeDef * hi2c)
{
    // TX Done .. Do Something!
}
```

پس از فراخوانی تابع فوق، دستگاه جانبی I2C شروع به ارسال تمام بایت های داده در بافر یک به یک می کند تا زمانی که انجام شود) در حالت (DMA وقتی این کار انجام شد، این تابع زیر فراخوانی و اجرا می شود، اگر می خواهید پس از تکمیل انتقال داده کاری انجام دهید، آن را به کد خود در فایل منبع برنامه (main.c) اضافه کنید

HAL_I2C_Master_Receive_DMA

```
HAL_I2C_Master_Receive_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size);
```

```
void HAL_I2C_MasterRxTxCpltCallback (I2C_HandleTypeDef * hi2c)
{
    // RX Done .. Do Something!
}
```

پس از فراخوانی تابع فوق، دستگاه جانبی I2C شروع به دریافت تمام بایت های داده ورودی در بافر می کند تا زمانی که کار تمام شود) در حالت (DMA وقتی این کار انجام شد، این تابع زیر فراخوانی و اجرا می شود، اگر می خواهید پس از تکمیل دریافت داده کاری انجام دهید، آن را به کد خود در فایل منبع برنامه (main.c) اضافه کنید.

AL_I2C_IsDeviceReady

AL_I2C_IsDeviceReady (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint32_t Trials, uint32_t Timeout);

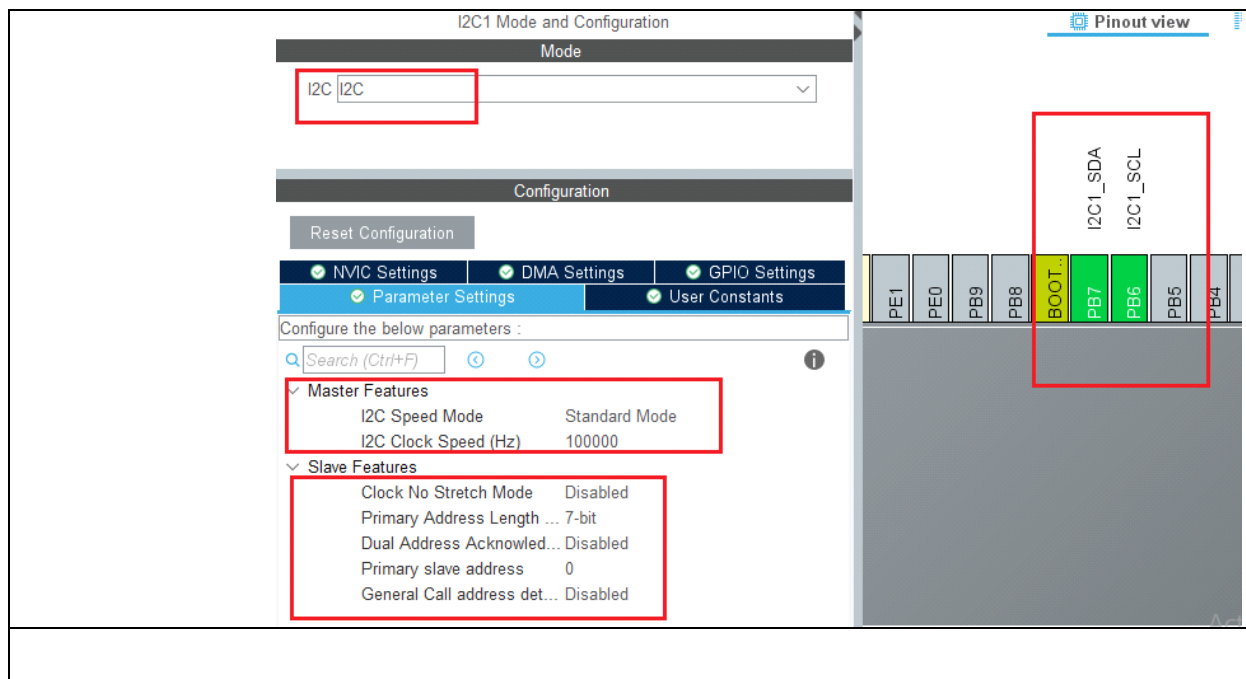
این یک ابزار مفید برای بررسی اینکه آیا یک دستگاه اسلیو در I2C واقعا وجود دارد و کار می کند یا خیر، است.

16.12 ارسال و دریافت داده برای دو میکرو در نقش های Master و Slave

16.12.1 تنظیمات مستر

برای اتصالات دو باس از دو میکرو را نظیر به نظیر متصل نموده و از مقاومت pullup هم استفاده نمایید.

ابتدا تنظیمات مستر را در stm32cubemx انجام دهید.



سپس کدها را مشابه ذیل ویرایش نمایید.

Master Code

```
#include "main.h"

I2C_HandleTypeDef hi2c1;
UART_HandleTypeDef huart1;

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_USART1_UART_Init(void);
```

```

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_I2C1_Init();
    MX_USART1_UART_Init();
    uint8_t dataToSend1[25]="master send 1403\n\r" ;

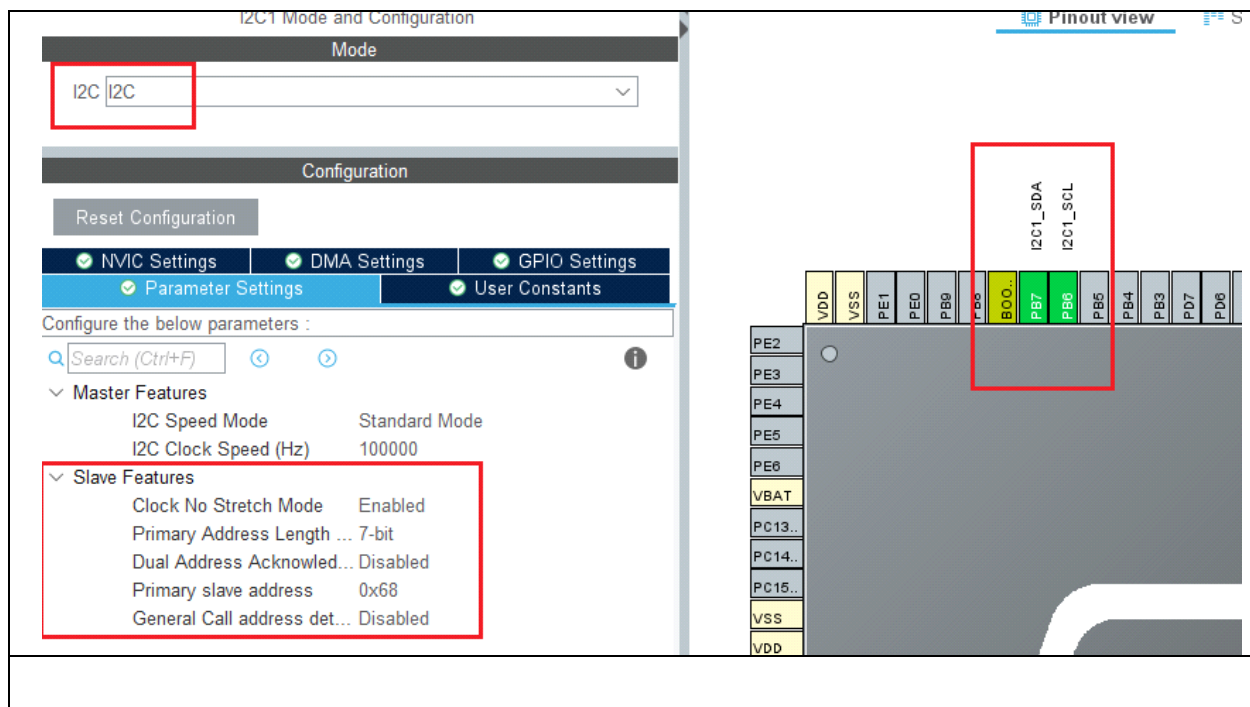
    while (1)
    {
        HAL_I2C_Master_Transmit(&hi2c1, 0x68 << 1, &dataToSend1[0], 25, HAL_MAX_DELAY);
        HAL_Delay(1000);
    }
}

```

16.12.2 تنظیمات اسلیو

16.12.2.1 دریافت داده به روش polling در اسلیو

تنظیمات اسلیو را در stm32cube مطابق شکل زیر انجام دهید. تنظیمات uart را هم انجام دهید.



کدهای ایجاد شده را در محیط keil ویرایش نمایید.

Slave code(polling)

```
#include "main.h"
```

```

I2C_HandleTypeDef hi2c1;

UART_HandleTypeDef huart1;
uint8_t receivedData[16]="*start*\n\r";


void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_USART1_UART_Init(void);

int main(void)
{

    HAL_Init();

    SystemClock_Config();

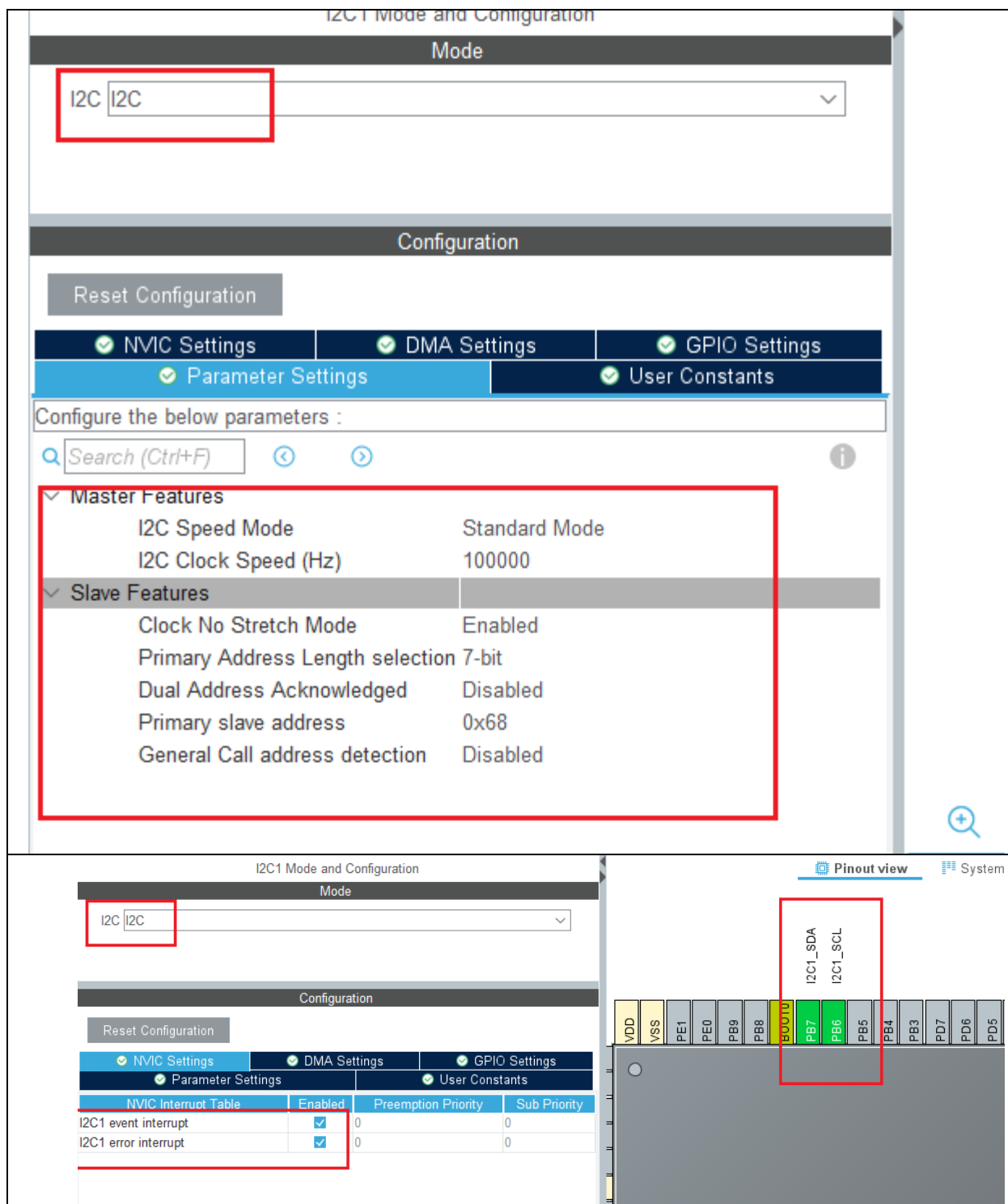
    MX_GPIO_Init();
    MX_I2C1_Init();
    MX_USART1_UART_Init();

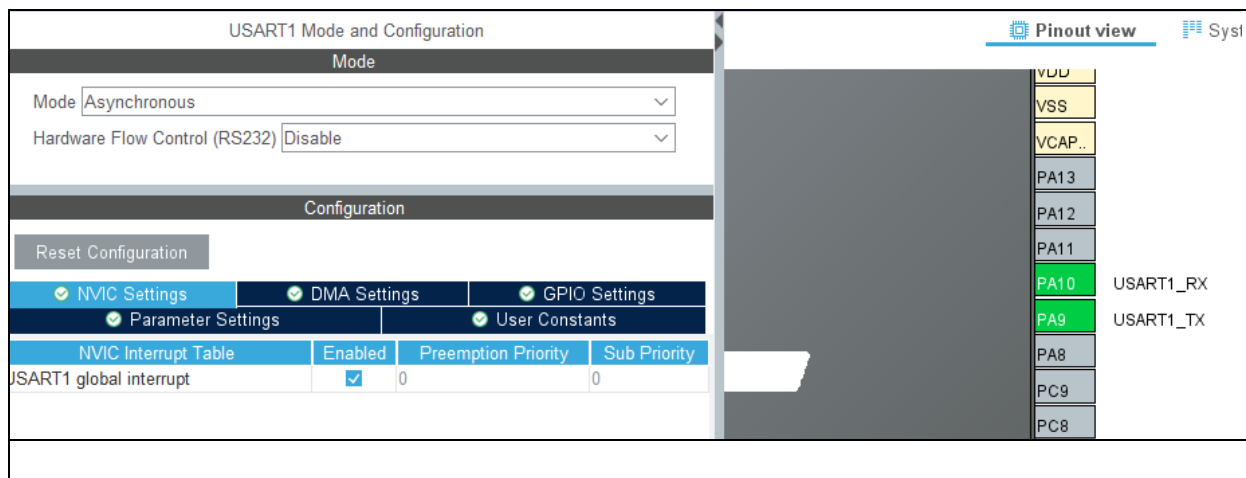
    HAL_UART_Transmit(&huart1, &receivedData[0], 25,HAL_MAX_DELAY);
    while (1)
    {

        HAL_I2C_Slave_Receive(&hi2c1, &receivedData[0], 25, HAL_MAX_DELAY);
        HAL_UART_Transmit(&huart1, &receivedData[0], 25,HAL_MAX_DELAY);
    }
}

```

16.12.2.2 دریافت داده به روش وقفه در اسلیو





Slave Code(interrupt)

```
#include "main.h"

#define zbufferSize      25

I2C_HandleTypeDef hi2c1;
UART_HandleTypeDef huart1;

uint8_t receivedData[zbufferSize];

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_USART1_UART_Init(void);

void HAL_I2C_SlaveRxCpltCallback(I2C_HandleTypeDef *hi2c) {

    HAL_UART_Transmit_IT(&huart1,receivedData, zbufferSize);
    HAL_I2C_Slave_Receive_IT(&hi2c1, receivedData, zbufferSize);

}

int main(void)
{

    HAL_Init();
```

```

SystemClock_Config();

MX_GPIO_Init();
MX_I2C1_Init();
MX_USART1_UART_Init();

    HAL_UART_Transmit(&huart1,"salam",5,10);
    HAL_I2C_Slave_Receive_IT(&hi2c1, receivedData, zbufferSize);
while (1)
{

}

}

```

16.12.2.3 دریافت داده به روش DMA در اسلیو

I2C1 Mode and Configuration

Mode

I2C I2C

Configuration

Reset Configuration

✓ NVIC Settings

✓ DMA Settings

✓ GPIO Settings

✓ Parameter Settings

✓ User Constants

Configure the below parameters :

Master Features

I2C Speed Mode

Standard Mode

I2C Clock Speed (Hz)

100000

Slave Features

Clock No Stretch Mode

Enabled

Primary Address Length selection

7-bit

Dual Address Acknowledged

Disabled

Primary slave address

0x68

General Call address detection

Disabled

Configuration

Reset Configuration

✓ Parameter Settings

✓ User Constants

✓ NVIC Settings

✓ DMA Settings

✓ GPIO Settings

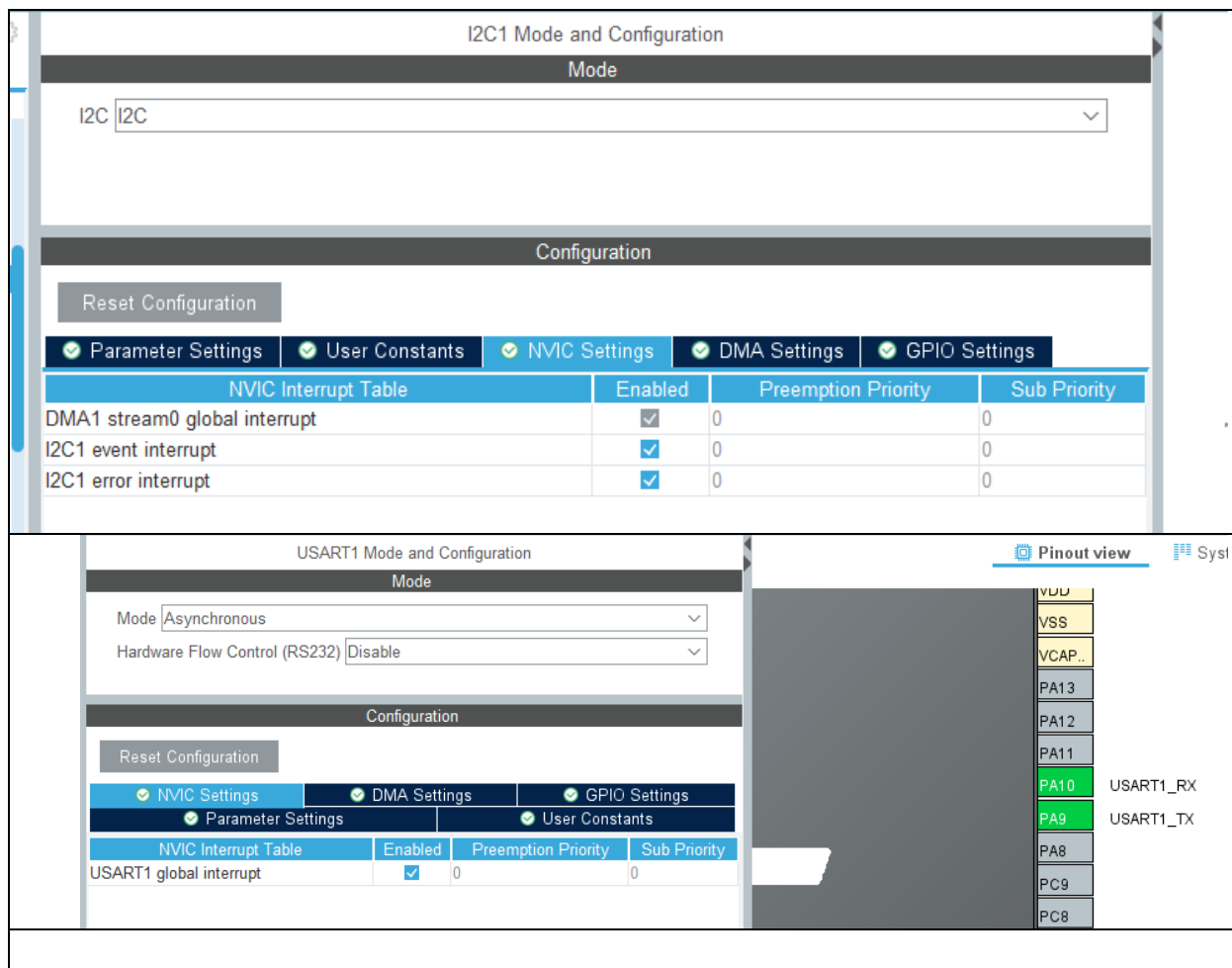
DMA Request	Stream	Direction	Priority
I2C1_RX	DMA1 Stream 0	Peripheral To Memory	Low

Add

Delete

DMA Request Settings

Peripheral		Memory
Mode	Normal	Increment Address ☐
Use Fifo	☐	Threshold
Data Width	Byte	Byte



Slave code(DMA)

```
#include "main.h"
```

```
#define zbufferSize 25
```

```
uint8_t receivedData[zbufferSize];
```

```
uint8_t dataIndex = 0;
```

```
I2C_HandleTypeDef hi2c1;
```

```
DMA_HandleTypeDef hdma_i2c1_rx;
```

```
UART_HandleTypeDef huart1;
```

```
void SystemClock_Config(void);
```

```

static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
static void MX_I2C1_Init(void);
static void MX_USART1_UART_Init(void);

void HAL_I2C_SlaveRxCpltCallback(I2C_HandleTypeDef *hi2c) {

    HAL_UART_Transmit_IT(&huart1,receivedData, zbuffersize);
    HAL_I2C_Slave_Receive_DMA(&hi2c1, receivedData, zbuffersize);

}

int main(void)
{

    HAL_Init();

    SystemClock_Config();

    MX_GPIO_Init();
    MX_DMA_Init();
    MX_I2C1_Init();
    MX_USART1_UART_Init();

    HAL_UART_Transmit(&huart1,"slave start",sizeof("slave start"),100);
    HAL_I2C_Slave_Receive_DMA(&hi2c1, receivedData, zbuffersize);
    while (1)
    {

    }

}

```

16.13 ارسال داده به حافظه EEPROM 24c32

تراشه C3224 یکی از انواع EEPROM (Electrically Erasable Programmable Read-Only Memory) است که به طور خاص برای ذخیره سازی داده ها در محصولات الکترونیکی طراحی شده است. در ادامه، به بررسی ویژگی ها، مشخصات و کاربردهای این تراشه می پردازیم.

16.13.1 مشخصات و ویژگی های 24c32

مشخصات و ویژگی های عبارتند از:

- ظرفیت: C3224 دارای ظرفیت 32 کیلوبیت (4 کیلو بایت) حافظه است که به صورت 256 کلمه 16 بیتی تقسیم بندی می شود.
 - پروتکل ارتباطی: این تراشه از پروتکل I²C (Inter-Integrated Circuit) برای ارتباط با میکروکنترلرها و دستگاه های دیگر استفاده می کند.
 - ولتاژ کاری: C3224 معمولاً در محدوده ولتاژ 2.5 ولت تا 5.5 ولت کار می کند، که آن را برای استفاده در بسیاری از مدارها و سیستم ها مناسب می سازد.
 - زمان نوشتن و خواندن: زمان نوشتن: تقریباً 5 تا 10 میلی ثانیه برای یک نوشتن کلمه.
 - زمان خواندن: معمولاً کمتر از 1 میلی ثانیه برای یک عملیات خواندن.
 - چرخه نوشتن و ذخیره سازی: تقبل حدود 1 میلیون چرخه نوشتن برای هر کلمه و حدود 100 سال زمان حفظ داده.
 - آدرس دهی: دارای آدرس دهی 8 بیتی برای انتخاب کلمات در حافظه است.
- تراشه های C3224 در چندین کاربرد مختلف استفاده می شوند، از جمله:

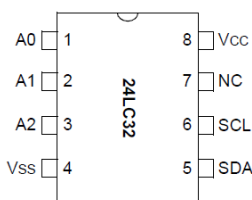
- تنظیمات کاربر: مثل ذخیره سازی تاریخ و ساعت، تنظیمات شبکه، و دیگر پارامترهای قابل تنظیم.
- دستگاه های الکترونیکی مصرفی: مانند ساعت های دیجیتال و لوازم الکترونیکی خانگی.
- سیستم های صنعتی: در کنترلرها و دستگاه های پایش.
- دستگاه های پزشکی: برای ذخیره داده های بیمار یا تنظیمات دستگاه ها.

از جمله مزایای این تراشه می‌توان به موارد ذیل اشاره کرد.

- حافظه غیر فرآر: داده‌ها حتی در صورت قطع برق نیز حفظ می‌شوند.
- حجم کوچک: ایده‌آل برای استفاده در برنامه‌های کم‌فضا.
- در دسترس بودن: در مقایسه با دیگر حافظه‌های غیر فرآر از قیمت معقولی برخوردار است.

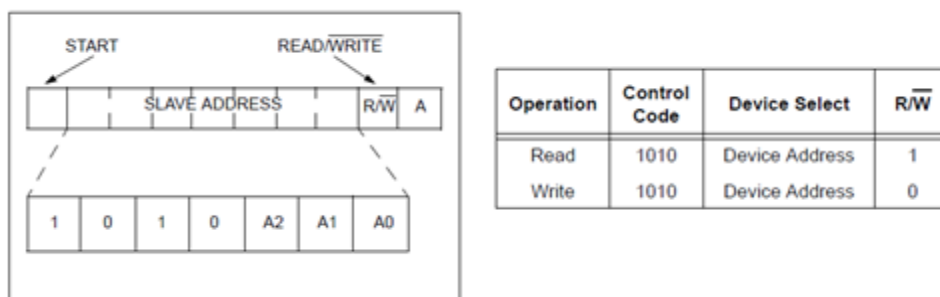
16.13.2 آدرس حافظه به عنوان Slave

تصویر حافظه در شکل نشان داده شده است.



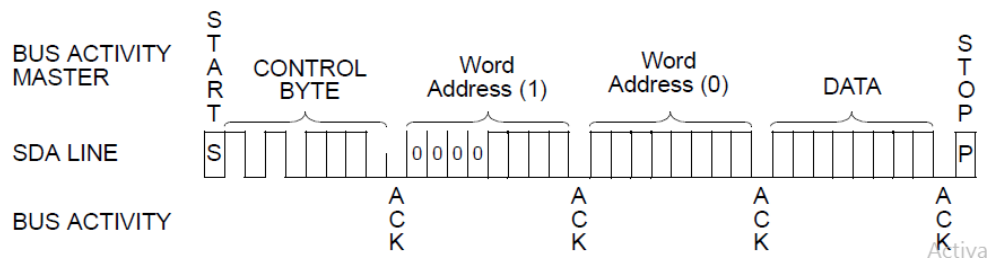
شکل 16-1: پایه‌های تراشه حافظه 24C32

آدرس Slave برای این تراشه به صورت سخت‌افزاری با اتصال پایه‌های A0 تا A2 به ولتاژ زمین یا Vcc مطابق شکل 16-2 تنظیم می‌گردد. به عنوان مثال اگر همه این پایه‌ها به زمین متصل شوند، آدرس Slave برابر با 0x50 خواهد بود. بر اساس این نحوه آدرس‌دهی حداکثر 8 تراشه‌ی حافظه از این مدل را می‌توان به گذرگاه I²C متصل نمود.



شکل 16-2: آدرس Slave برای حافظه EEPROM

در حالت کلی مانند شکل 16-3، پردازنده به عنوان Master پس از ارسال آدرس حافظه Slave مورد نظر (Control byte)، آدرس فضایی که قرار است عملیات خواندن یا نوشتن در آن انجام شود را ارسال نموده و سپس داده‌ها مبادله می‌شوند. حجم این حافظه 32KB است و فضای آدرس آن در محدوده‌ی 0x0000 تا 0x03FF قرار دارد. بنابراین آدرس داده درون حافظه با دو بایت نشان داده می‌شود.



شکل 16-3: بسته ارتباطی حافظه EEPROM با پروتکل I²C

16.13.3 زیربرنامه های مفید برای EEPROM 24C32

تراشه به عنوان یک حافظه EEPROM با ظرفیت مناسب و پروتکل ارتباطی ساده، گزینه‌ای ایده‌آل برای بسیاری از کاربردها در زمینه‌های مختلف الکترونیکی و صنعتی به شمار می‌آید.

برای کار با حافظه می‌توان از زیربرنامه های فایل جانبی استفاده کرد که اعم زیربرنامه ها عبارتند از:

: at24_isConnected

این تابع بررسی می‌کند که آیا EEPROM به درستی به میکروکنترلر متصل است یا نه. اگر با موفقیت به آدرس EEPROM دسترسی پیدا کند، true باز می‌گرداند و در غیر این صورت False.

: at24_write

این تابع داده را به آدرس مشخصی در EEPROM می‌نویسد. دارای قفل برای جلوگیری از مسابقه بین خواندن و نوشتن است. داده‌ها به تدریج در اندازه‌های معین به EEPROM ارسال می‌شوند تا از بزرگی داده‌ها خارج نشود. در صورت وقوع هر گونه مشکل یا زمان‌بر شدن، تابع false را برمی‌گرداند.

: at24_read

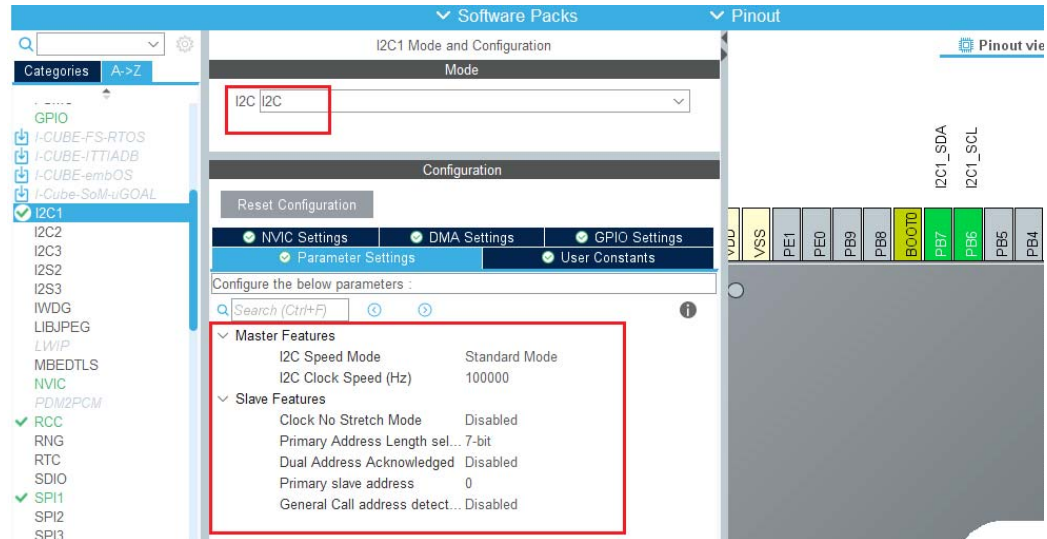
این تابع داده‌ها را از آدرس مشخصی در EEPROM می‌خواند. مشابه تابع نوشتن، از قفل برای جلوگیری از تصادف در خواندن و نوشتن استفاده می‌شود.

:at24_eraseChip

این تابع برای پاک کردن تمام محتوای EEPROM به کار می‌رود. داده‌های موجود در EEPROM با 0xFF جایگزین می‌شوند. این تابع به اندازه کل EEPROM تمرکز می‌کند و داده‌ها را به صورت دسته‌ای 32 بیتی پاک می‌کند.

16.13.4 تبادل داده با حافظه به وسیله I2C

برای تبادل داده خطوط کلاک و داده باید نظیر به نظیر به حافظه متصل شود و از مقاومت pullup هم به ازای هر خط استفاده شود. سپس تنظیمات لازم در محیط stm32cubemx مطابق شکل زیر انجام شود.



لازم به ذکر است که I2C به بوس APB1 متصل است. از آنجا که داده های خوانده شده در حافظه روی Uart نمایش داده می شود تنظیمات آن را لحاظ نمایید. کدهای تهیه شده را محیط keil مطابق ذیل ویرایش نمایید و فایل جانبی at24cxx.c را به پروژه اضافه نمایید.

```
#include "main.h"
#include "string.h"
#include "at24cxx.h"

I2C_HandleTypeDef hi2c1;
UART_HandleTypeDef huart1;

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART1_UART_Init(void);
static void MX_I2C1_Init(void);

#define MEM_ADDR 0x00u

bool writeStatus = false;
bool readStatus = false;
bool eraseStatus = false;
```

```

uint8_t wData[] = "Hello World 123";
uint8_t rData[25];

int main(void)
{
    char scr[50];

    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART1_UART_Init();
    MX_I2C1_Init();

    while (1)
    {

        if(at24_isConnected()){
            eraseStatus = at24_eraseChip();
            HAL_Delay(10);
            writeStatus = at24_write(MEM_ADDR,wData, 15, 100);
            HAL_Delay(100);
            readStatus = at24_read(MEM_ADDR,rData, 15, 100);
            HAL_Delay(10);

        }
        sprintf(scr,"rdata=%s\n\r",rData);
        HAL_UART_Transmit(&huart1, scr, strlen(scr),HAL_MAX_DELAY);
        HAL_Delay(3000);
    }
}

```